

Nguyen Manh Hiep

Software Engineer · Full-Stack

Hoang Mai, Ha Noi (Remote-ready) | Hiep.work@outlook.com | 0978 618 337 | linkedin.com/in/hiepnm2003
github.com/TolaHiep

Summary

Software Engineer focused on full-stack web development: UI, backend, database, deployment and testing. Recent work includes real-time systems, document OCR, admin dashboards, workflow automation, and AI/LLM integrations inside real product flows. Most recent project is **SenAI**, a real-time Vietnamese–English meeting interpreter I built in **three weeks** as the main developer on the project team (frontend, backend, CI/CD, deployment). It is live at app.senai.online and took **3rd Prize and the Vietnam AI Stars award at the Vietnam AI Innovation Challenge 2026 (AI Singapore)**.

Career Objective

I am interested in Software Engineer or Full-stack Software Engineer roles where I can contribute from technical design through implementation, deployment and product operations. Right now I am going deeper into real-time architecture and production LLM integration: model routing, evaluation, and latency and cost budgets. I work with short specs, code review and real, non-mocked tests so features can be checked by system behaviour, not only by how the code looks.

Selected Achievements

- **3rd Prize + Vietnam AI Stars**, Vietnam AI Innovation Challenge 2026 (AI Singapore), for SenAI — a real-time Vietnamese–English meeting interpreter shipped to public production.
- **3 systems delivered end-to-end** in ~12 months (SenAI, Notarex, ASC Farm) — each one carried from requirements through architecture, implementation, tests and deployment.
- **Blank repo to live production in 3 weeks** on SenAI — frontend, backend, database, security, CI/CD and deployment, on a spec-driven AI workflow (**16 design specs + 16 plans** kept in the repo).
- **697 automated / UAT test cases authored** across the three portfolio products (418 Notarex, 147 SenAI real non-mocked, 132 ASC Farm) — every AI-generated feature ships behind a real test.
- **Sub-400 ms live translation** (~75 ms ASR + ~295 ms MT measured end-to-end) with zero errors up to concurrency 8, and runtime provider switching with no restart and no downtime.

How I Work With AI

Toolchain: Claude Code (primary agent, subagents, custom skills), Cursor, ChatGPT, Gemini, GitHub Copilot; MCP servers (GitNexus code intelligence, browser automation) wired into the coding loop

Context engineering: Repo-level agent context (CLAUDE.md / AGENTS.md) pinning architecture, conventions, protocol source-of-truth and branch policy, so agents produce merge-ready code without re-briefing

Spec-driven loop: Brainstorm → versioned design spec (docs/specs/) → implementation plan (docs/plans/) → AI implementation → real test run; one canonical spec per system, never parallel versions

AI guardrails: Every AI-written feature lands behind real no-mock UAT (live API/WebSocket/LiveKit calls), call-graph impact analysis before editing a symbol, and CI on every push

Projects

SenAI — Real-time Vietnamese–English Meeting Interpreter (app.senai.online) 2026

Platform for live bilingual meetings — per-speaker ASR, translation and voice cloning, automatic minutes, on-prem or commercial model routing. Built for the Vietnam AI Innovation Challenge 2026 (AI Singapore) — **3rd Prize, Vietnam AI Stars Award and AI Singapore Award**. Main developer on the project team — the whole frontend, most of the backend behind it, the real-time meeting room, the translation pipeline, CI/CD and deployment.

- **Delivery:** blank repo to public production deployment in **3 weeks**, on a spec-driven loop (design spec → plan → agent implementation → real UAT) with 16 design specs and 16 implementation plans kept in the repo.
- **Frontend ownership:** the entire Next.js 16 / React 19 / Tailwind v4 app — 7 feature screens (landing, auth, Live, Rooms, Notes, Reports, admin), its own design system, bilingual VI–EN i18n (~370 keys) with a test guard against missing keys, in-app light/dark, PWA.
- **Real-time meeting room (LiveKit):** per-mic pipelines, camera and screen share, translated two-way chat, a waiting room with a host-controlled approval gate, mute/kick, speaker mode (grant/revoke publish rights), QR invites, and settings editable mid-meeting — the bot reloads config and restarts per-mic handlers in under a second.
- **Backend & data:** layered FastAPI service (routers / engine / store / model router); migrated the store from SQLite to PostgreSQL 16 + pgvector (psycopg3, pooling, migrate on boot) without changing public function signatures; meeting config/token/WS auth APIs; bilingual LLM minutes exported to MD/Word/PDF and emailed per recipient language via Amazon SES.

- **Real-time AI pipeline:** one independent ASR+MT pipeline per LiveKit mic track (no shared buffers), 2-lane fast→refine subtitles and a streaming *Cabin* mode that translates while the speaker is still talking. Measured **~75 ms ASR + ~295 ms MT (<400 ms end-to-end)**, zero errors up to concurrency 8.
- **Pluggable Model Router:** one capability contract (`asr/mt_fast/mt_refine/tts/llm`) behind self-hosted models (Qwen3, F5-TTS on vLLM) or commercial APIs (Google, Deepgram, OpenAI, ElevenLabs); pipeline, provider and keys switch at runtime from an admin UI with auto-fallback.
- **RAG & prompt engineering:** glossaries and uploaded documents chunked into pgvector embeddings, LLM term extraction, ASR phonetic correction, bilingual LLM summaries; prompt-injection defenses covered by a pentest suite.
- **Auth, security & integrations:** Google OAuth + email sign-in, JWT access/refresh rotation, rate limiting, per-meeting RBAC and audit logs; an on-prem-only mode blocking every cloud call for data residency; a public AI Gateway (`/ai/*`) with API keys for third-party callers.
- **CI/CD & QA (mine end-to-end):** GitHub Actions CI on every push, pull-based deploys to a VPS on Nginx + systemd + TLS, Docker for LiveKit and PostgreSQL; **37 UAT suites / 147 real (non-mocked) tests** plus a benchmark harness comparing commercial pipelines.

Notarex — Notary Document OCR & Digitization

2026

Digitizes notarial records — AI OCR from scanned papers, online editing, automatic Word generation. Role — full-stack, AI orchestration and infrastructure.

- Built configurable form templates, the AI OCR extraction flow and an edit/save/download workflow through JWT-secured OnlyOffice, with automatic DOCX generation via docxtempl.
- Designed a 3-role permission model (admin / director / employee) scoped per notary office for tenant isolation.
- Built a Celery + Redis job queue (ZSET pending queue, atomic active-counter, heartbeat reconciliation) that detects and recovers zombie OCR jobs under load; MinIO (S3) storage, Dockerized behind Nginx with GitHub Actions CI/CD.
- Delivered over 3.5 months and covered by **43 test modules / 418 automated test cases** adding a new document type became configuration rather than code.

ASC Farm — Agricultural Production Management (Web Admin + Mobile PWA)

2026

Production management for agriculture, built to stay reliable in low-connectivity fields.

- Built a headless Frappe v15 backend with an event-driven task engine (step / frequency / prerequisite), a worst-of status model, load-balanced assignment, REST API and 2-role auth — **verified by 132 passing tests**.
- Anti-fraud completion photos (in-app camera, GPS, watermark) plus an idempotent offline queue so field data is never lost or duplicated.
- Delivered a satellite heatmap (Leaflet + Esri), work calendar, notifications and step-by-step SOPs.

Education

Hanoi University of Science and Technology (HUST), BSc in Computer Science

2021 – 2026

- Coursework applied directly in production work — data structures & algorithms, databases, software engineering, computer networks, operating systems, machine learning.

Skills

AI-assisted development: Claude Code (agents, skills, hooks), Cursor, ChatGPT, Gemini, GitHub Copilot, MCP servers, prompt & context engineering, spec-driven AI workflows, RAG, LLM orchestration, vLLM self-hosting

Frontend: React 19, Next.js 16/14, TypeScript, Tailwind v4, Ant Design, Zustand, TanStack Query, PWA / offline-first, i18n

Backend: FastAPI, Node.js, NestJS / ExpressJS, Spring Boot, Python, Java, REST, WebSocket, WebRTC (LiveKit), Celery, Redis, JWT / OAuth, RBAC

Data: PostgreSQL 16 (+ pgvector), MySQL, MariaDB, SQLAlchemy 2.0 async, schema migrations, vector search & embeddings, MinIO (S3)

DevOps & Cloud: Docker & Docker Compose, Nginx, systemd, TLS, VPS deployment, GitHub Actions CI/CD, Cloudflare, Amazon SES

Engineering practice: Systems thinking, software architecture, SOLID, OOP, design patterns, real (non-mocked) integration & security testing, requirements analysis, SRS / Use Case / ERD

Languages: Vietnamese (native), English (technical reading, writing and documentation)